

Primeiro projeto - Física Computacional

Evandro Risso Venâncio¹

¹Instituto de Física de São Carlos - IFSC USP

risso.evandro@usp.br

Introdução

Este relatório tem como objetivo principal apresentar e explicar os códigos desenvolvidos no primeiro projeto da matéria de Introdução à Física Computacional, além de demonstrar a aplicação prática do uso do Fortran 77.

Exercício 1

Este programa tem como o objetivo, calcular a área e o volume de um torus, dados os raios interno e externo fornecidos pelo usuário.

Inicialmente, as variáveis *raio1* e *raio2* são declaradas como *real*, representando respectivamente os raios interno e externo do torus. A variável *pi* também é declarada como *real * 8* para garantir precisão no cálculo.

```
1  real raio1, raio2
2  real*8 pi
```

Logo após, o valor de *pi* é atribuído utilizando a função *acos*(-1D0), que retorna o valor de *pi* em dupla precisão, ou seja, com 15 casas decimais.

```
1  pi = acos(-1D0)
```

O programa então solicita ao usuário que insira os valores dos raios interno e externo do torus, utilizando os comandos *write* e *read* para interação com o terminal.

O comando *write* é utilizado para uma mensagem no terminal, que será formatado conforme a sua respectiva linha.

Por exemplo, *write*(*, 1) faz com que o programa 'pule' para a linha 1, onde a formatação da mensagem é definida. Em seguida, o *read* lê o valor escrito no terminal e armazena esse valor em sua respectiva variável.

```
1  write(*, 1); read(*,*) raio1
2  write(*, 2); read(*,*) raio2
3  1 format('Digite o valor do raio interno: ' $)
4  2 format('Digite o valor do raio externo: ' $)
```

Após a leitura dos valores, o programa calcula a área total do torus usando a fórmula $4 \times \pi^2 \times r1 \times r2$ e o volume usando $2 \times \pi^2 \times r1^2 \times r2$. Os resultados são exibidos no terminal.

```
1  write(*,*)'A area do torus e: ', 4*(pi**2)*raio1*raio2
2  write(*,*)'O volume do torus e: ', 2*(pi**2)*(raio1**2)*raio2
```

O código inteiro está representado abaixo:

```

1  program ex1
2      real raio1, raio2
3      real*8 pi
4
5      pi = acos(-1D0)
6
7      write(*, 1); read(*,*) raio1
8
9      write(*, 2); read(*,*) raio2
10 1  format('Digite o valor do raio interno: ' $)
11 2  format('Digite o valor do raio externo: ' $)
12
13      write(*,*)'A area do torus e: ', 4*(pi**2)*raio1*raio2
14      write(*,*)'O volume do torus e: ', 2*(pi**2)*(raio1**2)*raio2
15
16  end program ex1

```

Exercício 2

O exercício 2 tem como objetivo calcular a área lateral e o volume de um paralelepípedo de base triangular formado por três vetores v_1 , v_2 , e v_3 , fornecidos pelo usuário. (Os vetores foram criados com o *dimensions* do Fortran).

```

1  dimension v1(3), v2(3), v3(3)
2  write(*, *)'Digite os valores do vetor 1: '; read(*,*) v1
3  write(*, *)'Digite os valores do vetor 2: '; read(*,*) v2
4  write(*, *)'Digite os valores do vetor 3: '; read(*,*) v3

```

Para realizar esses cálculos, foi utilizado álgebra, como o produto vetorial e o produto escalar.

A imagem a seguir, representa a figura formada pelos 3 vetores:

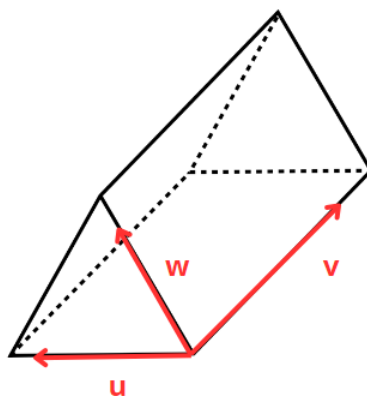


Figura 1. Paralelepípedo de base triangular formado pelos vetores

Para formar a base do paralelepípedo, a ideia é calcular a normal ao plano formado pelos vetores v_1 e v_2 . Isso é feito utilizando o produto vetorial:

$$\mathbf{v1} \times \mathbf{v2} = (v_{12} \cdot v_{23} - v_{13} \cdot v_{22})\hat{i} + (v_{13} \cdot v_{21} - v_{11} \cdot v_{23})\hat{j} + (v_{11} \cdot v_{22} - v_{12} \cdot v_{21})\hat{k}$$

Em Fortran o produto vetorial fica dessa forma:

```
1  !Produto vetorial entre v1 X v2
2  crossI = v1(2)*v2(3) - v1(3)*v2(2)
3  croosJ = v1(1)*v2(3) - v1(3)*v2(1)
4  crossK = v1(1)*v2(2) - v1(2)*v2(1)
```

Observação: Da forma que o código foi feito, o sinal do produto vetorial em $\hat{j}$ está invertido, porém ao decorrer do exercício esse sinal foi corrigido.

Esse produto vetorial nos dá um vetor perpendicular ao plano formado por $v1$ e $v2$. A norma deste vetor, ou seja, o comprimento, é igual a área do paralelogramo formado por $v1$ e $v2$. Se a norma for zero, isso significa que $v1$ e $v2$ são paralelos e, portanto, não formam uma base para o paralelepípedo.

```
1  norm_cross = sqrt(crossI**2 + croosJ**2 + crossK**2)
2
3  !Verificar se v1 e v2 sao paralelos
4  if (norm_cross .EQ. 0.0) then
5      write(*,*) 'Os vetores v1 e v2 sao paralelos'
6      stop
7  end if
```

O próximo passo agora é verificar se o vetor $v3$ está no mesmo plano formado por $v1$ e $v2$. Isso é feito calculando o produto escalar entre o vetor $v3$ e a normal ao plano $v1 \times v2$. Se o resultado for zero, $v3$ está no mesmo plano que o vetor $v1$ e $v2$ e, portanto, não pode formar um paralelepípedo. (O sinal da componente $\hat{j}$ foi corrigido).

```
1  !Verificar se v3 esta no mesmo plano de v1 e v2
2  dot = v3(1)*crossI + (-1*v3(2)*croosJ) + v3(3)*crossK
3  if (dot .EQ. 0.0) then
4      write(*,*) 'O vetor v3 esta no mesmo plano que v1 e v2'
5      stop
6  end if
```

Para calcular a área lateral do paralelepípedo, foi utilizado os produtos vetoriais dos vetores $v2 \times v3$ e $v3 \times v1$. A soma das normas desses vetores, junto com a norma do vetor $v1 \times v2$, dá a área lateral total das faces laterais do paralelepípedo.

```
1  crossV2I = v2(2)*v3(3) - v2(3)*v3(2)
2  crossV2J = v2(3)*v3(1) - v2(1)*v3(3)
3  crossV2K = v2(1)*v3(2) - v2(2)*v3(1)
4
5  crossV3I = v3(2)*v1(3) - v3(3)*v1(2)
6  crossV3J = v3(3)*v1(1) - v3(1)*v1(3)
7  crossV3K = v3(1)*v1(2) - v3(2)*v1(1)
8
9  normV1 = sqrt(crossV2I**2 + crossV2J**2 + crossV2K**2)
10 normV2 = sqrt(crossV3I**2 + crossV3J**2 + crossV3K**2)
11
12 area_lateral = 2*(normV1 + normV2 + norm_cross)
```

O volume do paralelepípedo é dado pelo valor absoluto do produto escalar entre o vetor $v3$ e o vetor normal ao plano $v1 \times v2$:

$$\text{Volume} = |\mathbf{v3} \cdot (\mathbf{v1} \times \mathbf{v2})|$$

```
1 volume = abs(dot)
```

O código completo do exercício está sendo ilustrado abaixo:

```
1 program ex2
2   dimension v1(3), v2(3), v3(3)
3   real crossI, croosJ, crossK, norm_cross, area_lateral, volume, dot
4   real crossV2I, crossV2J, crossV2K, crossV3I, crossV3J, crossV3K
5   real normV1, normV2
6
7   write(*, *) 'Digite os valores do vetor 1: '; read(*,*) v1
8   write(*, *) 'Digite os valores do vetor 2: '; read(*,*) v2
9   write(*, *) 'Digite os valores do vetor 3: '; read(*,*) v3
10
11  !Produto vetorial entre v1 X v2
12  crossI = v1(2)*v2(3) - v1(3)*v2(2)
13  croosJ = v1(1)*v2(3) - v1(3)*v2(1)
14  crossK = v1(1)*v2(2) - v1(2)*v2(1)
15
16  norm_cross = sqrt(crossI**2 + croosJ**2 + crossK**2)
17
18  !Verificar se v1 e v2 sao paralelos
19  if (norm_cross .EQ. 0.0) then
20      write(*,*) 'Os vetores v1 e v2 sao paralelos'
21      stop
22  end if
23
24  !Verificar se v3 esta no mesmo plano de v1 e v2
25  dot = v3(1)*crossI + (-1*v3(2)*croosJ) + v3(3)*crossK
26  if (dot .EQ. 0.0) then
27      write(*,*) 'O vetor v3 esta no mesmo plano que v1 e v2'
28      stop
29  end if
30
31  !Calcular produto vetorial de v2 X v3
32  crossV2I = v2(2)*v3(3) - v2(3)*v3(2)
33  crossV2J = v2(3)*v3(1) - v2(1)*v3(3)
34  crossV2K = v2(1)*v3(2) - v2(2)*v3(1)
35
36  !Calcular produto vetorial de v3 X v1
37  crossV3I = v3(2)*v1(3) - v3(3)*v1(2)
38  crossV3J = v3(3)*v1(1) - v3(1)*v1(3)
39  crossV3K = v3(1)*v1(2) - v3(2)*v1(1)
40
41  !Calcular a norma dos vetores
42  normV1 = sqrt(crossV2I**2 + crossV2J**2 + crossV2K**2)
43  normV2 = sqrt(crossV3I**2 + crossV3J**2 + crossV3K**2)
44
45  !Calcular a area lateral
46  area_lateral = 2*(normV1 + normV2 + norm_cross)
47
48  !Calcular o volume
49  volume = abs(dot)
50
51  write(*,*) 'A area lateral e: ', area_lateral
52  write(*,*) 'O volume e: ', volume
```

Exercício 3

Neste exercício, o objetivo é ler uma série de números reais de um arquivo de entrada (do tipo `REAL*8` = dupla precisão = 15 casas decimais), determinar a quantidade total de números presentes (N) e, em seguida, permitir que o usuário especifique um valor M (onde $M \leq N$). O programa então vai ordenar os M menores números desses N números e salvá-los em um arquivo de saída.

Para ordenar os números, foi utilizado um algoritmo de ordenação chamado **Bubble Sort**. Este algoritmo compara elementos em uma lista e os troca se estiverem fora de ordem, comparando se o número atual é maior que o próximo. Esse processo é repetido até que a lista esteja ordenada.

A figura 3 mostra como funcionaria esse algoritmo.

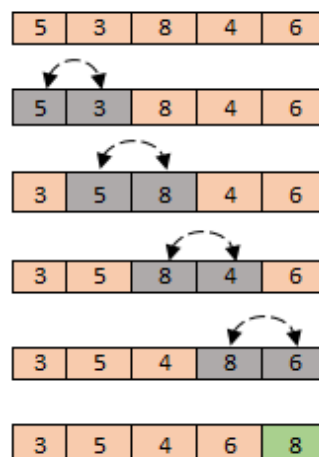


Figura 2. Representação do Bubble Sort. Fonte: Betrybe - Bubble Sort: Tudo sobre.

O código desenvolvido em FORTRAN começa com a alocação de memória para armazenar os números lidos do arquivo e a abertura dos arquivos de entrada e de saída:

```
1  real*8, allocatable :: numbers(:)
2  real*8 temp
3  integer m, n, i
4
5  open(unit=1, file='entrada-3-#14668849.in')
6  open(unit=2, file='saida-3-#14668849.out')
```

Após isso é necessário determinar o número total de elementos N no arquivo de entrada. Isso é feito através de um loop que lê todos os números do arquivo, incrementando um contador a cada leitura.

```
1  !Numero de valores no arquivo
2  n = 0
3  do
4      read(1, *, end=100)
```

```

5     n = n + 1
6 end do
7
8 100 continue

```

Logo após determinar N , o programa aloca memória para armazenar esses números e lê os valores novamente, desta vez armazenando-os em um array.

```

1 !Alocar memoria para os numeros - Array
2 allocate (numbers (n))
3
4 !Voltar para o inicio do arquivo
5 rewind(1)
6
7 !Ler os valores do arquivo
8 do i = 1, n
9     read(1, *) numbers(i)
10 end do

```

O exercício solicita também que o usuário que insira o valor de M (sendo menor ou igual a N). Caso contrário, o programa exibe uma mensagem de erro e interrompe a execução.

```

1 !Ler o valor de M
2 write(*, 130); read(*, *) m
3
4 !Verificar se M e menor ou igual ao numero de valores
5 if (m > n) then
6     print*, 'M deve ser menor ou igual ao numero de valores'
7     stop
8 end if

```

Em seguida, o programa ordena os M menores números utilizando o algoritmo de Bubble Sort.

```

1 !Ler e ordenar os primeiros M numeros menores do arquivo
2 do i = 1, n - 1
3     do j = 1, n - 1
4         if (numbers(j) > numbers(j + 1)) then
5             temp = numbers(j)
6             numbers(j) = numbers(j + 1)
7             numbers(j + 1) = temp
8         end if
9     end do
10 end do

```

Finalmente, os M menores números são escritos no arquivo de saída, juntamente com o valor de M .

```

1 !Escrever os M numeros menores no arquivo de saida
2 write(2, *) 'Valor de M = ', m
3 do i = 1, m
4     write(2, '(F11.9)') numbers(i)
5 end do
6
7 write(*, 120) n
8
9 120 format('O numero de valores do arquivo e: ' i5)

```

```

10 130 format('Digite o valor de M: ' $)
11
12     close(1); close(2)
13 end program ex3

```

Para melhor entendimento o arquivo de entrada tem esse formato:

```

1 0.396608114
2 0.897073984
3 3.46368861
4 3.09318447
5 1.08731675
6 5.91860580
7 4.18480444
8 2.22633338
9 1.73413491
10 4.19433928
11 10.2843132
12 3.55715179
13 7.05034208
14 14.0054150
15 13.4899273
16 7.52578259

```

E o arquivo de saída esse formato:

```

1 Valor de M = 3
2 0.396608114
3 0.552463055
4 0.703183413

```

Com a primeira linha mostrando a quantidade de números que devem aparecer ordenados. Por fim, o código completo do exercício está representado abaixo:

```

1 program ex3
2   real*8, allocatable :: numbers(:)
3   real*8 temp
4   integer m, n, i
5
6   open(unit=1, file='entrada-3-#14668849.in')
7   open(unit=2, file='saida-3-#14668849.out')
8
9   !Numero de valores no arquivo
10  n = 0
11  do
12     read(1,*, end=100)
13     n = n + 1
14  end do
15
16 100 continue
17
18  !Alocar memoria para os numeros - Array
19  allocate(numbers(n))
20
21  !Voltar para o inicio do arquivo
22  rewind(1)
23

```

```

24 !Ler os valores do arquivo
25 do i = 1, n
26     read(1,*) numbers(i)
27 end do
28
29 !Ler o valor de M
30 write(*, 130); read(*,*) m
31
32 !Verificar se M e menor ou igual ao numero de valores
33 if (m > n) then
34     print*, 'M deve ser menor ou igual ao numero de valores'
35     stop
36 end if
37
38 !Ler e ordenar os primeiros M numeros menores do arquivo
39 do i = 1, n - 1
40     do j = 1, n - 1
41         if (numbers(j) > numbers(j + 1)) then
42             temp = numbers(j)
43             numbers(j) = numbers(j + 1)
44             numbers(j + 1) = temp
45         end if
46     end do
47 end do
48
49 !Escrever os M numeros menores no arquivo de saida
50 write(2, *) 'Valor de M = ', m
51 do i = 1, m
52     write(2, '(F11.9)') numbers(i)
53 end do
54
55 write(*, 120) n
56
57 120 format('O numero de valores do arquivo e: ' i5)
58 130 format('Digite o valor de M: ' $)
59
60 close(1); close(2)
61 end program ex3

```

Exercício 4

A ideia desse exercício é calcular o valor de $\cos(x)$ utilizando a série de Taylor:

$$\cos(x) = 1 - \frac{1}{2!}x^2 + \frac{1}{4!}x^4 - \frac{1}{6!}x^6 + \dots$$

Essa série expande a função cosseno em uma soma infinita de termos, onde cada termo envolve potências de x e fatoriais, alternando sinais. A precisão do cálculo é determinada por um valor de tolerância ϵ , que define o critério de parada da soma dos termos. Na primeira parte da tarefa, o programa deve calcular o valor de $\cos(x)$ com uma precisão $\epsilon = 10^{-5}$, utilizando a série de Taylor. Esse resultado é então comparado com o valor obtido pela função intrínseca $\cos(x)$ do FORTRAN.

Já na segunda parte, o programa foi modificado para trabalhar em dupla precisão, utilizando a função intrínseca $\text{dcos}(x)$ para comparação. A precisão foi aumentada para

$\epsilon = 10^{-17}$, que representa a máxima precisão atingida.

O código começa solicitando ao usuário o valor de x . A precisão inicial é definida para $1.0D^{-5}$ para a primeira parte, e as variáveis necessárias para o cálculo da série são inicializadas:

```
1  real*8 x, precision, cosSeries, term, pi
2  integer n
3  precision = 1.0D-5
4
5  write(*, 100); read(*,*) x
```

O cálculo do cosseno é realizado utilizando a série de Taylor, onde cada termo é calculado e somado até que o valor absoluto do termo seja menor que a precisão definida:

```
1  precision = 1.0D-5
2  cosSeries = 1.0D0
3  term = 1.0D0
4  n = 0
5  factorial = 1
6  pi = acos(-1.0D0)
7
8  x = mod(x, 2.0D0*pi)
9
10 do while (abs(term) > precision)
11     n = n + 2
12     factorial = factorial * n * (n - 1)
13     term = (-1)**(n/2) * (x**n) / factorial
14     cosSeries = cosSeries + term
15 end do
```

No início do programa, as variáveis são inicializadas da seguinte forma:

- `precision` é definida como $1.0D^{-5}$ para a primeira parte, representando a precisão desejada no cálculo de $\cos(x)$.
- `cosSeries` é inicializada com o valor 1.0, que é o primeiro termo da expansão de Taylor para $\cos(x)$.
- `term` também é inicializada com 1.0, representando o primeiro termo da série.
- `n` começa em 0 e é usada como contador para as potências de x nos termos subsequentes da série.
- `factorial` é inicializada em 1 e será usada para calcular os fatoriais necessários para o denominador dos termos da série.
- `pi` é calculada utilizando a função intrínseca `acos(-1.0D0)`, que retorna o valor de π .

Antes de iniciar o cálculo da série, o valor de x é normalizado para o intervalo $[0, 2\pi]$ utilizando a função `mod`, uma vez que o cosseno é uma função periódica com período 2π . O cálculo da série de Taylor é realizado em um loop `do while`, que continua somando termos à série enquanto o valor absoluto do termo atual for maior que a precisão definida. A cada iteração:

- O valor de n é incrementado em 2 para avançar para o próximo termo da série, que alterna entre potências pares de x .
- O valor do fatorial necessário para o denominador do termo é calculado multiplicando o fatorial atual pelos dois próximos números subsequentes.

- O termo da série é calculado utilizando a fórmula $\text{term} = (-1)^{n/2} \times \frac{x^n}{n!}$, onde o sinal alternado é gerado por $(-1)^{n/2}$, e o termo é então somado ao valor acumulado de `cosSeries`.

Ao final do loop, o valor calculado para $\cos(x)$ pela série de Taylor é exibido e comparado com o valor obtido pela função intrínseca `cos(x)` do FORTRAN.

```
1  write(*, *) 'Cos (serie): ', cosSeries
2  write(*, *) 'Cos: ', cos(x)
```

Para a segunda parte do exercício, apenas o valor do ϵ foi alterado para chegar à máxima precisão entre os dois cossenos.

```
1  precision = 1.0D-17
```

O código final está abaixo:

```
1  program ex4
2      real*8 x, precision, cosSeries, term, pi
3      integer n
4
5      !Ler o valor de X
6      write(*, 100); read(*,*) x
7
8      precision = 1.0D-17
9      cosSeries = 1.0D0
10     term = 1.0D0
11     n = 0
12     factorial = 1
13     pi = acos(-1.0D0)
14
15     x = mod(x, 2.0D0*pi)
16
17     do while (abs(term) > precision)
18         n = n + 2
19         factorial = factorial * n * (n - 1)
20         term = (-1)**(n/2) * (x**n) / factorial
21         cosSeries = cosSeries + term
22     end do
23
24 100 format('Digite o valor de x: ' $)
25
26     write(*, *) 'Cos (serie): ', cosSeries
27     write(*, *) 'Cos: ', dcoss(x)
28
29 end program ex4
```

Exercício 5

O objetivo deste exercício é criar um programa que leia de um arquivo de entrada as permutações de N inteiros $(1, 2, \dots, N)$ e suas correspondentes paridades (-1 ou 1). O programa deve então produzir todas as permutações de $(N + 1)$ números, juntamente com a paridade correspondente de cada permutação.

O código foi desenvolvido para ler o maior número presente no arquivo de entrada e, com base nesse número, calcular todas as permutações possíveis dos números de 1 a $N + 1$,

incluindo a paridade correspondente de cada permutação.

Inicialmente, um vetor chamado *perm* é criado com espaço para armazenar até 20.000 elementos. Esse vetor será utilizado para gerar as permutações. Após a criação do vetor, os arquivos de entrada e saída são abertos para leitura e escrita, respectivamente.

Para identificar o maior valor na lista de permutações fornecida no arquivo de entrada, uma variável chamada *ibigger* é inicialmente definida como 1. Em seguida, o programa percorre todos os números do arquivo de entrada, verificando se o número atual (*icurrent*) é maior do que o valor armazenado em *ibigger*. Se for, o valor de *ibigger* é atualizado com o número atual. Esse processo garante que *ibigger* conterá o maior número presente no arquivo.

```
1 integer perm(20000), n
2
3 open(unit=1, file='entrada-5-#14668849.in')
4 open(unit=2, file='saida-5-#14668849.out')
5
6 ! Pegar maior numero
7 ibigger = 1
8 do
9   read(1, *, end=100) icurrent
10  if (icurrent .gt. ibigger) then
11    ibigger = icurrent
12  end if
13 end do
14 100 continue
```

Depois de determinar o valor de N (que é o maior número encontrado no arquivo), o código gera as permutações de $N + 1$ números. Para isso, o valor de *ibigger* é incrementado em 1, resultando no novo valor de $N + 1$.

```
1 ! Numero maximo para as permutacoes
2 ibigger = ibigger + 1
```

A parte principal do programa é a sub-rotina *permute*, que é uma função recursiva responsável por gerar todas as permutações de $N + 1$ elementos e calcular a paridade de cada uma delas. A paridade de uma permutação é determinada pelo número de inversões na ordem dos elementos:

- a inversão ocorre quando um número maior aparece antes de um número menor na permutação.
- Se o número total de inversões for par, a paridade é 1.
- Se o número total de inversões for ímpar, a paridade é -1 .

A sub-rotina *permute* funciona da seguinte forma:

- Se $n = 1$, significa que a permutação atual foi completamente gerada. Neste ponto, o programa calcula a paridade da permutação atual e escreve a permutação, junto com sua paridade, no arquivo de saída.
- Se $n > 1$, a sub-rotina chama a si mesma recursivamente para gerar todas as permutações possíveis dos elementos restantes. A cada passo, ela troca elementos da permutação para garantir que todas as combinações sejam cobertas. Dependendo do valor de n , ela realiza as trocas de elementos de duas maneiras diferentes para garantir a correta geração das permutações.

```

1 call permute(perm, ibigger, ibigger, permute)
2 close(2)
3 close(1)
4 end program ex5
5
6 !Funcao recursiva para a permutacao
7 subroutine permute(perm, n, original_n, dumperm)
8   integer perm(*), n, original_n
9   external dumperm
10  integer i, temp, paridade
11
12  if (n .EQ. 1) then
13    paridade = 1
14    do i = 1, original_n - 1
15      do j = i + 1, original_n
16        if (perm(i) .GT. perm(j)) then
17          paridade = -paridade
18        end if
19      end do
20    end do
21    write(2, '(10i2, i2)') (perm(i), i = 1, original_n), paridade
22  else
23    do i = 1, n
24      call dumperm(perm, n - 1, original_n, dumperm)
25      if (mod(n, 2) .EQ. 0) then
26        temp = perm(i)
27        perm(i) = perm(n)
28        perm(n) = temp
29      else
30        temp = perm(1)
31        perm(1) = perm(n)
32        perm(n) = temp
33      end if
34    end do
35  end if
36  return
37 end subroutine permute

```

Resumindo:

- O programa começa identificando o maior número no arquivo de entrada para definir o valor de N .
- Esse valor é então incrementado em 1 para gerar as permutações de $N+1$ números.
- A sub-rotina recursiva *permute* é chamada para gerar todas as possíveis permutações dos números de 1 a $N + 1$.
- A cada permutação gerada, o programa calcula sua paridade com base no número de inversões na ordem dos elementos.
- As permutações e suas respectivas paridades são escritas no arquivo de saída.

Para o arquivo de entrada contendo o maior número igual a $ibigger = 3$ o programa produz todas as permutações de 4 números e suas respectivas paridades. O arquivo de saída contém as permutações de 1, 2, 3, 4 com as paridades calculadas conforme mostrado no trecho abaixo:

```

1  1  2  3  4-1
2  2  1  3  4  1
3  3  1  2  4-1
4  1  3  2  4-1
5  2  3  1  4  1
6  3  2  1  4-1
7  4  2  3  1  1
8  2  4  3  1  1
9  3  4  2  1-1
10 4  3  2  1  1
11 2  3  4  1  1
12 3  2  4  1-1
13 4  1  3  2-1
14 1  4  3  2-1
15 3  4  1  2  1
16 4  3  1  2-1
17 1  3  4  2-1
18 3  1  4  2  1
19 4  1  2  3  1
20 1  4  2  3  1
21 2  4  1  3-1
22 4  2  1  3  1
23 1  2  4  3  1
24 2  1  4  3-1

```

Infelizmente, os itens (b) e (c) deste exercício, que consistiam em calcular o determinante de uma matriz $N \times N$ utilizando as permutações e em resolver um sistema de equações lineares, não foram realizados devido a dificuldades encontradas durante a implementação.

Portanto, o código final do exercício contempla apenas o item (a), que ficou estruturado da seguinte forma:

```

1  program ex5
2      integer perm(20000), n
3
4      open(unit=1, file='entrada-5-#14668849.in')
5      open(unit=2, file='saida-5-#14668849.out')
6
7      ! Pegar maior numero
8      ibigger = 1
9      do
10         read(1, *, end=100) icurrent
11         if (icurrent .gt. ibigger) then
12             ibigger = icurrent
13         end if
14     end do
15 100 continue
16
17     ! Numero maximo para as permutacoes
18     ibigger = ibigger + 1
19
20     !Tamanho da matriz
21     n = ibigger
22
23     !Permutacao inicial

```

```

24     do i = 1, ibigger
25         perm(i) = i
26     end do
27
28     call permute(perm, ibigger, ibigger, permute)
29     close(2)
30     close(1)
31 end program ex5
32
33 !Funcao recursiva para a permutacao
34 subroutine permute(perm, n, original_n, dumperm)
35     integer perm(*), n, original_n
36     external dumperm
37     integer i, temp, paridade
38
39     if (n .EQ. 1) then
40         paridade = 1
41         do i = 1, original_n - 1
42             do j = 1 + 1, original_n
43                 if (perm(i) .GT. perm(j)) then
44                     paridade = -paridade
45                 end if
46             end do
47         end do
48         write(2, '(10i2, i2)') (perm(i), i = 1, original_n), paridade
49     else
50         do i = 1, n
51             call dumperm(perm, n - 1, original_n, dumperm)
52             if (mod(n, 2) .EQ. 0) then
53                 temp = perm(i)
54                 perm(i) = perm(n)
55                 perm(n) = temp
56             else
57                 temp = perm(1)
58                 perm(1) = perm(n)
59                 perm(n) = temp
60             end if
61         end do
62     end if
63     return
64 end subroutine permute

```

Exercício 6

Neste sexto exercício o objetivo é calcular o volume V_d de uma esfera em d dimensões, utilizando a função `rand()` do FORTRAN, que gera números reais pseudo-aleatórios entre 0 e 1. O programa calcula esse volume de duas maneiras: através de um método numérico utilizando Monte Carlo e através de uma expressão teórica. Os resultados foram comparados e analisados para dimensões $d = 2, 3$ e 4 , variando o número M de pontos aleatórios.

O código desenvolvido consiste em um programa principal que solicita ao usuário as entradas necessárias (dimensão, número de pontos aleatórios e o raio da esfera) e utiliza essas informações para calcular o volume da esfera tanto pelo método Monte Carlo quanto pela fórmula teórica. O volume calculado por ambos os métodos é então exibido

ao usuário.

Abaixo, uma descrição detalhada do código:

```
1 program ex6
2   integer m, d
3   real*8 volume, volumeTeorico, raio
4
5   write(*, *) 'Digite a dimensao: '; read(*, *) d
6   write(*, *) 'Digite o numero de pontos: '; read(*, *) m
7   write(*, *) 'Digite o raio: '; read(*, *) raio
8
9   call volume_esfera_mc(d, m, raio, volume)
10  call volume_esfera_teorico(d, raio, volumeTeorico)
11  write(*, *) 'Volume (rand): ', volume
12  write(*, *) 'Volume Teorico: ', volumeTeorico
13
14 end program ex6
```

No programa principal, as variáveis são definidas para armazenar o número de pontos aleatórios (M), a dimensão da esfera (d) e o raio da esfera (r). O usuário insere esses valores, que são então utilizados para calcular o volume da esfera pelos dois métodos:

- **Monte Carlo:**

- O volume é calculado utilizando uma sub-rotina que gera M pontos aleatórios dentro de um cubo de lado $2r$.
- A sub-rotina conta quantos desses pontos estão dentro da esfera de raio r .
- O volume da esfera é então estimado como a fração desses pontos dentro da esfera multiplicada pelo volume do cubo.

- **Fórmula Teórica:**

- A fórmula teórica para o volume de uma esfera em d dimensões é utilizada.
- Essa fórmula é baseada na expressão:

$$V_d = \frac{\pi^{d/2}}{\Gamma(1 + d/2)} R^d$$

A função *Gamma* é aproximada numericamente no código. A sub-rotina *volume_esfera_mc* é responsável por calcular o volume da esfera usando o método Monte Carlo.

```
1 subroutine volume_esfera_mc(d, m, raio, volume)
2   integer d, m, n_dentro, i, j
3   real*8 volume, x, soma_quadrados, raio
4   n_dentro = 0
5   do i = 1, m
6     soma_quadrados = 0.0d0
7     do j = 1, d
8       x = rand()
9       soma_quadrados = soma_quadrados + (x)**2
10    end do
11    if (soma_quadrados .LE. 1.0d0) then
12      n_dentro = n_dentro + 1
13    end if
14  end do
15
16  volume = (2.0d0*raio)**d * (n_dentro / real(m))
17 end subroutine volume_esfera_mc
```

Para cada um dos M pontos gerados, o código calcula a soma dos quadrados das coordenadas, verificando se esse ponto está dentro da esfera (ou seja, se a soma dos quadrados das coordenadas é menor ou igual a 1). O volume é então calculado como a fração de pontos dentro da esfera multiplicada pelo volume do cubo em que os pontos foram gerados.

A sub-rotina *volume_esfera_teorico* calcula o volume teórico da esfera utilizando a fórmula mencionada.

```

1 subroutine volume_esfera_teorico(d, raio, volumeTeorico)
2   integer d
3   real*8 raio, volumeTeorico, pi, gamma_d, ad
4
5   ad = d
6   pi = acos(-1D0)
7   ad = ad/2.0d0 + 1.0d0
8   gamma_d = 1
9
10  do while (ad .gt. 1.D0)
11    ad = ad - 1
12    gamma_d = gamma_d * ad
13  end do
14
15  volumeTeorico = (pi**(d / 2) / gamma_d) * (raio**d)
16 end subroutine volume_esfera_teorico

```

A sub-rotina calcula a função Gamma numericamente para valores inteiros e seminteiros, que é necessária para determinar o volume teórico. A função Gamma é aproximada como um produto de termos decrescentes, começando em $1 + d/2$ e terminando em 1.

Os volumes calculados tanto pelo método Monte Carlo quanto pela fórmula teórica são exibidos ao usuário, permitindo a comparação.

O código final do exercício está representada abaixo:

```

1 program ex6
2   integer m, d
3   real*8 volume, volumeTeorico, raio
4
5   write(*, *) 'Digite a dimensao: '; read(*, *) d
6   write(*, *) 'Digite o numero de pontos: '; read(*, *) m
7   write(*, *) 'Digite o raio: '; read(*, *) raio
8
9   call volume_esfera_mc(d, m, raio, volume)
10  call volume_esfera_teorico(d, raio, volumeTeorico)
11  write(*, *) 'Volume (rand): ', volume
12  write(*, *) 'Volume Teorico: ', volumeTeorico
13
14 end program ex6
15
16 subroutine volume_esfera_mc(d, m, raio, volume)
17   integer d, m, n_dentro, i, j
18   real*8 volume, x, soma_quadrados, raio
19   n_dentro = 0
20   do i = 1, m
21     soma_quadrados = 0.0d0
22     do j = 1, d
23       x = rand()

```

```

24     soma_quadrados = soma_quadrados + (x)**2
25     end do
26     if (soma_quadrados .LE. 1.0d0) then
27         n_dentro = n_dentro + 1
28     end if
29 end do

30
31     volume = (2.0d0*raio)**d * (n_dentro / real(m))
32 end subroutine volume_esfera_mc
33
34 subroutine volume_esfera_teorico(d, raio, volumeTeorico)
35     integer d
36     real*8 raio, volumeTeorico, pi, gamma_d, ad
37
38     ad = d
39     pi = acos(-1D0)
40     ad = ad/2.0d0 + 1.0d0
41     gamma_d = 1
42
43     do while (ad .gt. 1.D0)
44         ad = ad - 1
45         gamma_d = gamma_d * ad
46     end do
47
48     volumeTeorico = (pi**(d / 2) / gamma_d) * (raio**d)
49 end subroutine volume_esfera_teorico

```

Exercício 7

O objetivo deste exercício é calcular os volumes V_d de esferas em diferentes dimensões, variando de 0 a d , onde d é fornecido pelo usuário. O programa deve receber como entrada o valor do raio R e a dimensão d , calcular os volumes e armazená-los em um arquivo de saída. Em seguida, os volumes calculados devem ser utilizados para gerar um gráfico de V_d como função de d , variando d de 0 a 25 e considerando diferentes valores de raio ($R = 0.9, 1.0$ e 1.1).

O código consiste em um programa principal que recebe como entrada a dimensão máxima d e o raio R . O programa calcula os volumes para todas as dimensões de 1 a d e armazena os resultados em um arquivo de saída. Além disso, outra sub-rotina é responsável por calcular os volumes para d variando de 0 a 25 e gerar um arquivo de saída para criação dos gráficos.

```

1 program ex7
2     integer d
3     real*8 volume, raio
4
5     open(1, file='saida-7-#14668849.out')
6
7     write(*, *) 'Digite a dimensao: '; read(*, *) d
8     write(*, *) 'Digite o raio: '; read(*, *) raio
9
10    do i = 1, d
11        call volume_esfera_teorico(i, raio, volume)
12        write(1, *) 'Dimensao: ', i, ' Volume: ', volume
13    end do

```

```

14
15     close(1)
16
17     call grafico()
18 end program ex7

```

O programa principal solicita ao usuário a dimensão máxima d e o raio R e, em seguida, calcula os volumes para todas as dimensões de 1 até d . Esses volumes são calculados usando a sub-rotina *volume_esfera_teorico*, que implementa a fórmula teórica para o cálculo do volume de uma esfera em d dimensões.

A sub-rotina *volume_esfera_teorico* é responsável pelo cálculo do volume teórico de uma esfera em uma determinada dimensão d , utilizando a fórmula:

$$V_d = \frac{\pi^{d/2}}{\Gamma(1 + d/2)} R^d$$

```

1 subroutine volume_esfera_teorico(d, raio, volumeTeorico)
2     integer d
3     real*8 raio, volumeTeorico, pi, gamma_d, ad
4
5     ad = d
6     pi = acos(-1D0)
7     ad = ad/2.0d0 + 1.0d0
8     gamma_d = 1
9
10    do while (ad .gt. 1.D0)
11        ad = ad - 1
12        gamma_d = gamma_d * ad
13    end do
14
15    volumeTeorico = (pi**(d / 2) / gamma_d) * (raio**d)
16 end subroutine volume_esfera_teorico

```

A sub-rotina utiliza a função Gamma aproximada para valores inteiros e seminteiros, necessária para calcular o volume teórico de uma esfera em d dimensões, como no exercício anterior. A sub-rotina *grafico* calcula os volumes para dimensões de 0 a 25, considerando os diferentes valores de raio ($R = 0.9, 1.0$ e 1.1). Os resultados são armazenados em um arquivo de saída, que pode ser utilizado para gerar gráficos utilizando o software Xmgrace.

```

1 subroutine grafico()
2     integer i
3     real*8 result
4
5     open(2, file='saida-7b-#14668849.out')
6
7     do i = 0, 25
8         call volume_esfera_teorico(i, 0.9d0, result)
9         call volume_esfera_teorico(i, 1.0d0, result)
10        call volume_esfera_teorico(i, 1.1d0, result)
11        write(2, *) i, result
12    end do
13

```

```

14  close(2)
15  end subroutine grafico

```

O programa principal gera um arquivo de saída com os volumes calculados para as dimensões de 1 até d , conforme exemplificado abaixo:

1	Dimensao:	1	Volume:	2.0000000000000000
2	Dimensao:	2	Volume:	3.1415926535897931
3	Dimensao:	3	Volume:	4.1887902047863905

A sub-rotina de gráfico gera um segundo arquivo de saída com volumes calculados para dimensões de 0 a 25, que pode ser utilizado para plotar gráficos no Xmgrace:

1	0	1.0000000000000000
2	1	2.2000000000000002
3	2	3.8013271108436504
4	3	5.5752797625706876
5	4	7.2250439018174664
6	5	8.4773848447991611
7	6	9.1549184203381078
8	7	9.2072322398828987
9	8	8.7002098971982935
10	9	7.7777114509556808
11	10	6.6144687504500244
12	11	5.3755682543339889
13	12	4.1906265641522999
14	13	3.1437374371369633
15	14	2.2757060528191020
16	15	1.5933832465550501
17	16	1.0813378893615306
18	17	0.71258481565809217
19	18	0.45672433720138195
20	19	0.28513347142490852
21	20	0.17361586051857106
22	21	0.10322719953680648
23	22	5.9997334314699446E-002
24	23	3.4121769754411273E-002
25	24	1.9005791125734750E-002
26	25	1.0376640674992674E-002

O gráfico resultante ficou da seguinte forma:

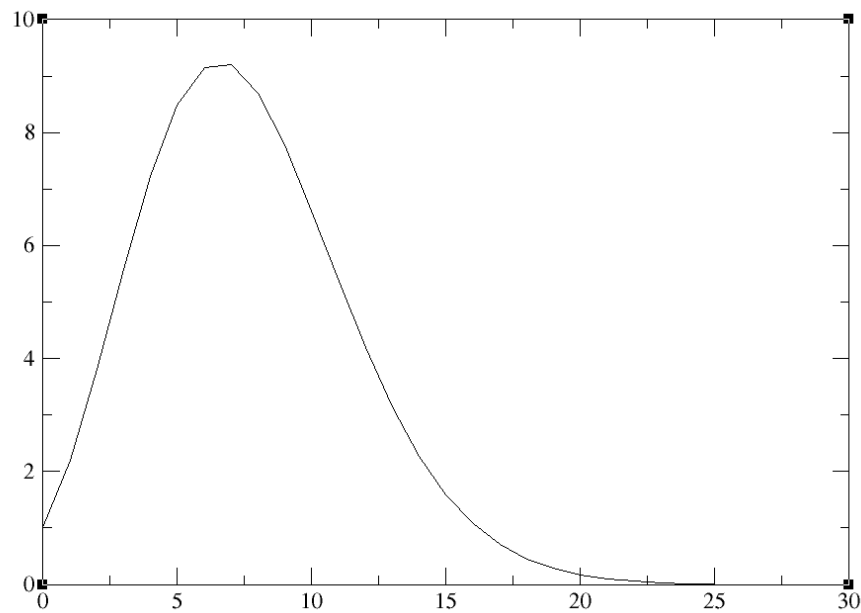


Figura 3. Gráfico dos volumes gerados

Por fim, o código completo está abaixo:

```

1  program ex7
2      integer d
3      real*8 volume, raio
4
5      open(1, file='saida-7-#14668849.out')
6
7      write(*, *) 'Digite a dimensao: '; read(*, *) d
8      write(*, *) 'Digite o raio: '; read(*, *) raio
9
10     do i = 1, d
11         call volume_esfera_teorico(i, raio, volume)
12         write(1, *) 'Dimensao: ', i, ' Volume: ', volume
13     end do
14
15     close(1)
16
17     call grafico()
18 end program ex7
19
20 subroutine volume_esfera_teorico(d, raio, volumeTeorico)
21     integer d
22     real*8 raio, volumeTeorico, pi, gamma_d, ad
23
24     ad = d
25     pi = acos(-1D0)
26     ad = ad/2.0d0 + 1.0d0
27     gamma_d = 1
28
29     do while (ad .gt. 1.D0)
30         ad = ad - 1
31         gamma_d = gamma_d * ad

```

```

32     end do
33
34     volumeTeorico = (pi**(d / 2) / gamma_d) * (raio**d)
35 end subroutine volume_esfera_teorico
36
37 subroutine grafico()
38     integer i
39     real*8 result
40
41     open(2, file='saida-7b-#14668849.out')
42
43     do i = 0, 25
44         call volume_esfera_teorico(i, 0.9d0, result)
45         call volume_esfera_teorico(i, 1.0d0, result)
46         call volume_esfera_teorico(i, 1.1d0, result)
47         write(2, *) i, result
48     end do
49
50     close(2)
51 end subroutine grafico

```

Exercício 8

Parte (a): Comparação entre o Volume de um Cubo e uma Esfera em d Dimensões

O volume de um cubo em d dimensões com lado 1 m é dado por:

$$V_{\text{cubo}} = 1^d = 1 \text{ m}^d$$

Enquanto o volume de uma esfera em d dimensões com raio $R = 1 \text{ m}$ é dado por:

$$V_{\text{esfera}} = \frac{\pi^{d/2}}{\Gamma(1 + d/2)} \times 1^d$$

onde Γ é a função Gamma, que generaliza o conceito de fatorial para números reais. A razão entre o volume do cubo e o volume da esfera em d dimensões é:

$$\text{Razão} = \frac{V_{\text{cubo}}}{V_{\text{esfera}}} = \frac{1}{\frac{\pi^{d/2}}{\Gamma(1+d/2)}}$$

À medida que a dimensão d tende ao infinito ($d \rightarrow \infty$), o fator $\frac{\pi^{d/2}}{\Gamma(1+d/2)}$ cresce rapidamente, resultando em uma esfera que ocupa uma fração cada vez menor do volume do cubo. Isso implica que a razão entre o volume do cubo e o volume da esfera cresce exponencialmente, tendendo ao infinito conforme $d \rightarrow \infty$.

Parte (b): Número de Avogadro em d Dimensões

O problema envolve o cálculo da ordem de magnitude do número de Avogadro em um mundo d -dimensional. O volume de uma proteína nesse espaço é dado por $1 \mu\text{m}^D$, e o volume de um átomo é dado por $1 \text{ \AA}^d$. O volume típico de um objeto macroscópico é 1 mm^d . Expressando todos os volumes na mesma unidade temos:

- $1 \mu\text{m}^D = 10^{-6d} \text{ m}^d$

- $1 \text{ \AA}^d = 10^{-10d} \text{ m}^d$
- $1 \text{ mm}^d = 10^{-3d} \text{ m}^d$

O número de átomos que cabem em um objeto macroscópico em d dimensões, equivalente ao número de Avogadro neste contexto, é:

$$N_{\text{Avogadro},d} = \frac{\text{Volume macroscópico}}{\text{Volume do átomo}} = \frac{10^{-3d} \text{ m}^d}{10^{-10d} \text{ m}^d} = 10^{7d}$$

Para $d = 3$, isso retorna o valor conhecido do número de Avogadro, aproximadamente 10^{23} . À medida que d aumenta, o número de Avogadro cresce exponencialmente, o que significa que em dimensões superiores, a quantidade de átomos necessários para preencher um volume macroscópico cresce.